

BACKEND TRACK - CAPSTONE REPORT

Embeddable Widget & Lead-Capture Platform

A hardened, multi-tenant FastAPI platform with Supabase persistence, cross-origin delivery, resilient enrichment, and verifiable acceptance evidence.

LIVE ON SUPABASE

9 TESTS PASSING

\$0 RUNTIME AI

Prepared from the implemented repository, deterministic test evidence, and live Supabase smoke verification.

Report date: 29 September 2026

1. Executive summary

The system turns one widget definition into a safe public acquisition path and a tenant-isolated owner view.



Outcome. The repository now implements authenticated widget management, versioned delivery, a public CORS submission boundary, abuse controls, two-provider geolocation fallback, a durable notification outbox, and owner analytics. PostgreSQL is provided by Supabase through an SSL-required Session pooler connection; the browser receives no database credential.

Verification. Nine deterministic tests pass. A live smoke test returned 200 for config and the second-origin demo, stored an idempotent lead in Supabase, and exposed it through the authenticated dashboard with correct short-cache headers.

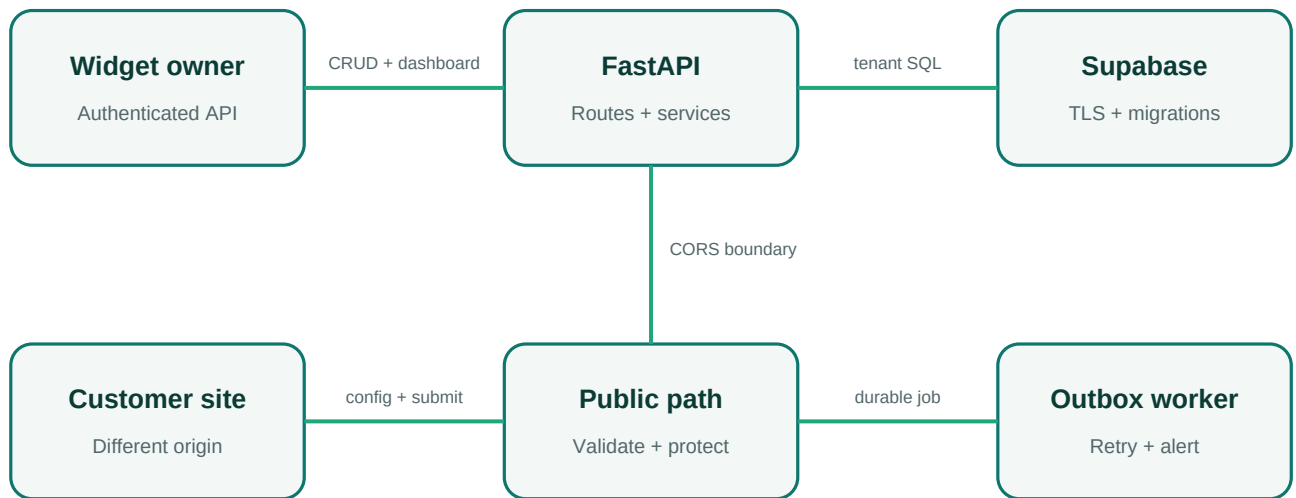
Engineering posture. The implementation favors clear failure boundaries over feature volume: untrusted input receives clean 4xx responses, non-critical providers may fail without losing leads, and repeated submissions do not duplicate storage or side effects.

Delivered scope

Area	Delivered behavior	Primary concern
Owner API	Tenant-scoped CRUD, snippet generation, lead list, aggregate stats	Authorization
Widget delivery	Versioned JS, short-lived config cache, Shadow DOM rendering	Performance
Public capture	CORS, preflight, size/schema checks, dynamic field validation	Boundary safety
Resilience	Rate limit, honeypot, geo fallback, durable notification retry	Graceful degradation
Persistence	Supabase migrations, indexes, transactions, idempotency key	Correctness

2. Architecture and request paths

The code and runtime keep authenticated administration separate from public delivery and visitor submission.



Three paths stay separate: owner administration, public delivery, and visitor submission.

Layer boundaries

Layer	Responsibility	Representative modules
HTTP	Routes, middleware, status codes, CORS, response headers	app/api.py
Business	Validation, rate limiting, enrichment, job processing	app/services/*
Data	Tenant-scoped SQL, migrations, analytics, outbox transaction	app/repositories.py
Client	Config fetch, Shadow DOM form, idempotent submission wiring	assets/widget.v1.js
Operations	Container runtime, Supabase TLS, DNS, seed and proof commands	docker-compose.yml

The Supabase connection stays server-side. This preserves multi-statement transactions, partial unique indexes, and `FOR UPDATE SKIP LOCKED` for the worker while avoiding exposure of secret or database credentials to the widget.

3. Data model and tenant isolation

Ownership is explicit in every persistent record and every authenticated query.

Table	Key fields	Integrity and access rule
tenants	id, name, api_key_hash	Only SHA-256 key hashes are stored; raw keys remain environment/user input.
widgets	tenant_id, type, form_fields, display_options	Every owner read/write filters tenant_id; public reads require active=true.
submissions	tenant_id, widget_id, data, geo, idempotency_key	Tenant is copied from the widget; partial unique index prevents retry duplicates.
side_effect_jobs	submission_id, status, attempts, next_attempt_at	Created in the same transaction as the lead; claimed with row locking.
ai_usage	provider, model, tokens, cost_usd	Reserved for attributed usage; current runtime AI cost is zero.

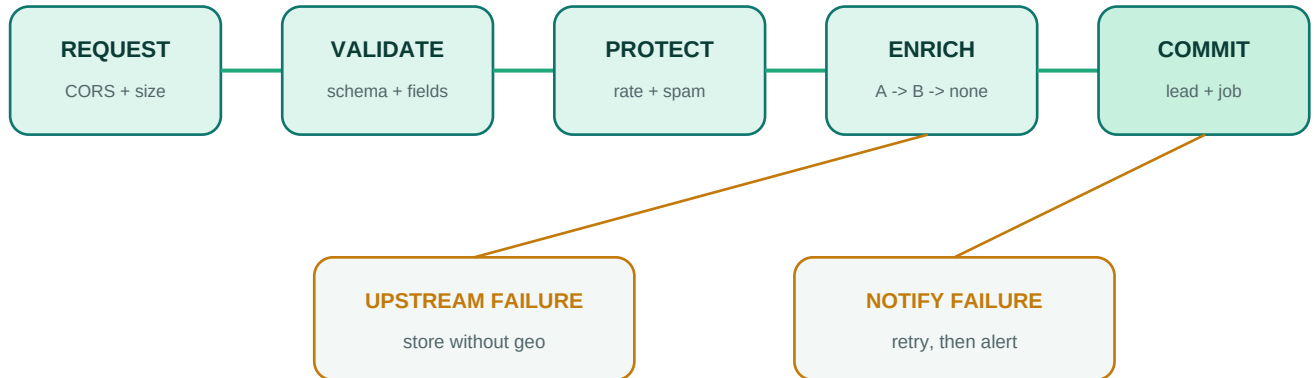
Isolation proof

Control	Proof
Authentication required	Requests without a valid API key return 401.
Opaque cross-tenant behavior	Tenant B receives 404 for tenant A widget reads and updates.
Submission ownership	Tenant identity is derived from the selected widget, never trusted from the public payload.
Dashboard scope	Tenant B receives an empty lead list after a tenant A submission.
Secret hygiene	.env is ignored; browser-safe configuration never contains the database URL.

The API accepts X-API-Key or a bearer key. Authentication returns only tenant identity and all repository methods require that identity as an explicit argument, reducing the chance of an unscoped query.

4. Hardened submission path

Every public request crosses explicit validation, abuse, enrichment, and persistence boundaries.



Stage	Successful path	Failure behavior
Transport	CORS preflight accepted from external origins	Unsupported or oversized input returns JSON 4xx
Validation	Configured fields normalized and type checked	Unexpected, missing, invalid email, or overlong fields return 422
Abuse	Per-IP and per-widget windows admit normal traffic	Burst returns 429 + Retry-After; honeypot returns quiet 202 and stores nothing
Enrichment	Provider A, then provider B	Both providers down: continue without geo
Persistence	Lead and outbox job commit atomically	Idempotency conflict returns original lead
Notification	Worker completes console notification	Exponential retry; final attempt emits alert

The main success invariant is simple: if validation and abuse checks pass, an optional dependency cannot turn the submission into a 500 or erase a stored lead.

5. API and delivery contract

A small surface area supports the three actors without mixing responsibilities.

Method	Path	Auth	Contract
GET	/health	No	Service health
POST	/api/widgets	Yes	Create widget
GET	/api/widgets	Yes	List tenant widgets
GET/PUT/DELETE	/api/widgets/{id}	Yes	Tenant-scoped CRUD
GET	/api/widgets/{id}/snippet	Yes	One-line versioned embed
GET	/assets/widget.v1.js	No	One-year immutable cache
GET	/widgets/{id}/config	No	60-second public config cache
POST	/submissions	No	Protected public capture
GET	/api/submissions	Yes	Tenant lead table
GET	/api/dashboard/stats	Yes	Time, widget, and geo aggregates

Embed contract

```
<script async src="http://localhost:8000/assets/widget.v1.js?id={widget-id}"></script>
```

Caching strategy

Asset	Header	Reason
widget.v1.js	public, max-age=31536000, immutable	Filename changes on release; safe long cache.
widget config	public, max-age=60, stale-while-revalidate=300	Owner edits become visible quickly without a fetch on every view.

The demo page runs on port 5500 while the API runs on 8000, proving that delivery and submission operate across different browser origins.

6. Verification and acceptance evidence

Automated and live checks cover the full contract, including failure paths.

9

automated tests

all passing

200

live config + demo

Supabase smoke

1

idempotent lead

visible in dashboard

Test area	Verified behavior
CRUD and isolation	Auth rejection, full CRUD, cross-tenant 404 behavior
Delivery	Versioned snippet, immutable bundle, config cache, different-origin wiring
CORS	OPTIONS preflight and allowed request headers
Boundary validation	Malformed JSON, invalid models, dynamic fields, 413 payload limit
Abuse and idempotency	429 with Retry-After, honeypot drop, replay without duplicate
Geo resilience	Provider A down -> B; both down -> submission continues
Notification resilience	Forced failure keeps lead and marks job for retry
Supabase configuration	URL precedence, missing config failure, forced TLS

Live Supabase transcript

```
ConfigStatus=200
ConfigCache=public, max-age=60, stale-while-revalidate=300
SubmissionAccepted=true
SubmissionReplayed=false
DashboardTotal=1
DemoStatus=200
```

The smoke test uses a stable idempotency key, so repeating it cannot create duplicate leads.

7. Operations, setup, and limitations

The runtime is deliberately small: one API container, one static demo container, and Supabase.

Operator runbook

Step	Command or action	Expected result
1	Set SUPABASE_DATABASE_URL to Session pooler URI	PostgreSQL URI, port 5432, password URL-encoded
2	<code>docker compose up --build</code>	API and demo containers start; migration runs
3	<code>docker compose exec app python seed.py</code>	Two demo tenants and fixed demo widget created
4	<code>docker compose exec app pytest -q</code>	9 tests pass
5	Open <code>localhost:8000/docs</code> and <code>localhost:5500</code>	API docs and cross-origin demo available

Operational notes

- TLS is required for every database connection.
- Docker Compose pins public DNS resolvers because Docker Desktop's embedded resolver failed on this host.
- The old local database container was removed after Supabase verification; its Docker volume was retained for recovery.
- The notification side effect is a console implementation by design; reliability behavior, not email delivery, is the evaluated concern.

Known limitations

- Rate-limit state is process-local; multi-replica deployment should use a shared store such as Redis.
- CORS defaults to all origins for embeddability; production should support an owner-configured allowlist where appropriate.
- Geo enrichment is best effort and private/test IPs may produce no location.
- The dashboard is a proof interface, not a production identity or session system.

8. Conclusion and references

The capstone meets its core contract and demonstrates production-oriented backend reasoning.

The implemented platform is intentionally compact but complete. It proves the hard parts of an embeddable lead product: multi-tenant ownership, public-browser compatibility, attacker-aware boundaries, failure isolation, durable side effects, observable evidence, and a real managed database.

The strongest interview narrative is not the widget's CSS. It is the submission invariant: validate early, reject abuse predictably, degrade optional dependencies, commit the business record once, and move secondary work behind a durable boundary.

Final status

Category	Status
Section 6 core requirements	PASS - all 15 mapped to proof
Shared backend requirements	PASS - architecture, validation, jobs, persistence, idempotency, secrets, cost
Automated verification	PASS - 9 tests
Live managed persistence	PASS - Supabase migration, seed, submit, dashboard
Second-origin delivery	PASS - demo page on port 5500

References

- FlyRank Internship Backend Track - Embeddable Widget & Lead-Capture Platform capstone brief.
- Repository README.md, DESIGN.md, EVIDENCE.md, BUILDLOG.md, capstone.yaml, and source modules.
- Supabase documentation: Connect to Postgres and API key security guidance.
- Docker documentation: Docker Desktop with the WSL 2 backend on Windows.

Prepared as a portfolio-ready implementation report. No secrets, passwords, or private connection strings are included.